Automated Drum Transcription

Recommendation for a drum-stem-to-MIDI pipeline

Context: stem separation, tempo detection and click-grid estimation are already available. The remaining requirement is the most reliable automatic drum transcription (ADT) engine for MIDI generation, preferably open-source and otherwise available under a perpetual or pay-as-you-go commercial model.

Primary recommendation

Use **ADTOF-pytorch** as the classification engine, but retrieve its raw per-frame activations and perform peak picking, grid association, velocity estimation and MIDI writing in your own pipeline. This is the strongest combination of deployability, reproducibility and practical transcription quality among currently usable open implementations.

Commercial fallback

Use **Superior Drummer 3 Tracker** as a locally licensed reference and fallback, particularly when multitrack or component drum stems are available. For a single mixed drum stem, validate it against your corpus before purchase.

Decision summary

Option	Recommendation	Reason
ADTOF-pytorch	**Primary**	Pretrained, local, programmable, strong published benchmark performance, direct access to model activations.
Superior Drummer 3 Tracker	**Paid fallback**	Mature correction workflow and perpetual desktop licensing; strongest when inputs are multitrack.
Klangio Drum2Notes	**External benchmark**	Useful hosted comparator and pay-as-you-go route, but a black box with uploaded audio and no raw activations.
ADT_STR / newer papers	**Watch list**	Promising research, but current public releases are not as deployable as a pretrained production inference engine.

Prepared 28 July 2026

1. Why ADTOF-pytorch is the best starting point

ADTOF-pytorch is a PyTorch port of the ADTOF Frame_RNN system for automatic drum-audio-to-MIDI transcription. It packages pretrained weights and exposes both a command-line interface and a Python API. The repository reports an F-measure of 88.51% on MDBDrums++, close to the original ADTOF implementation at 88.74%.

Operational strengths

- Runs locally and can be integrated directly into an existing Python audio pipeline.
- Uses a small dependency set: PyTorch, librosa, pretty_midi and NumPy.
- Bundles model weights and supports CPU or CUDA inference.
- Can return raw model activations instead of forcing the built-in MIDI post-processing path.
- Exposes per-class thresholding, enabling corpus-specific calibration.

Model output

The stock implementation predicts five broad General MIDI drum classes: kick (35), snare (38), tom (47), hi-hat (42) and cymbal (49). Its default post-processor runs at 100 frames per second, applies per-class thresholds and exports fixed-velocity MIDI notes.

Class	GM note	Default threshold
Kick	35	0.22
Snare	38	0.24
Tom	47	0.32
Hi-hat	42	0.22
Cymbal	49	0.30

Key design decision

Do not make the stock MIDI exporter the system boundary. Treat ADTOF as a probabilistic classifier whose activations feed your own timing and MIDI logic. Your accurate tempo and click detection are valuable priors that the standard exporter does not exploit.

2. Recommended integration architecture

Retrieve activations

The PyTorch port supports `return_activations=True`, returning a tensor shaped approximately as batch x time x five classes. Use the audio stem at its natural timing; do not pre-quantize the waveform.

```
from adtof_pytorch import transcribe_to_midi activations = transcribe_to_midi(
drum_stem_path, temporary_midi_path, return_activations=True, device="cuda", )
```

Recommended processing sequence

1	**Run inference**	Generate unquantized class activations from the isolated drum stem.
2	**Detect candidate peaks**	Apply class-specific peak picking. Preserve simultaneous peaks across classes for combinations such as kick plus crash.
3	**Associate with your grid**	For each detected event, find the nearest beat subdivision from the known tempo map. Snap only when the timing error is below a controlled tolerance.
4	**Preserve expressive timing**	Leave events unsnapped when they are outside tolerance so flams, drags, fills and deliberately loose placement are retained.
5	**Estimate velocity**	Derive velocity from local stem energy, activation height, or a calibrated combination. Avoid the stock fixed velocity of 100.
6	**Write MIDI**	Use your existing tempo map and generate deterministic General MIDI output with explicit note mapping and provenance metadata.

Starting parameters

Parameter	Initial value	Purpose
Peak-search window	+/- 20-30 ms	Find the strongest local class response near a candidate onset.
Grid-snap tolerance	30-45 ms	Correct small timing errors without flattening expressive placement.
Same-class refractory	20-35 ms	Suppress duplicate detections while preserving fast rudiments where possible.
Activation-to-velocity	Corpus-calibrated	Map activation and/or local RMS to useful MIDI velocity ranges.

Alternative onset-first architecture

If your existing detector provides accurate individual drum-hit candidates, let it determine **when** events occur and let ADTOF determine **what** occurred. For every candidate onset, sample the maximum activation for each class in a short local window, then apply independently calibrated class thresholds. This reduces duplicate onset errors and makes direct use of your strongest existing component.

3. Limitations and mitigation

Five-class vocabulary

ADTOF combines all toms into one class, ride and crash into one cymbal class, and open and closed hi-hat into one hi-hat class. This is the principal functional limitation for detailed notation or drum replacement.

- Start with reliable five-class MIDI and evaluate whether the downstream application genuinely needs finer articulation.
- Add a second-stage classifier around detected cymbal and hi-hat events to split ride/crash and open/closed hi-hat.
- Use spectral centroid, decay duration, channel balance and local component separation as secondary features.
- Treat tom pitch assignment as a separate ranking problem using spectral peak and temporal context.

Ghost notes and fast passages

Lower thresholds improve recall but can create bleed-triggered false positives. Tune thresholds per class and, if possible, per recording domain. Snare ghost notes generally require a lower threshold and a velocity floor, while cymbal classes often benefit from stricter temporal suppression.

4. Licensing and commercial use

> **Commercial-use caution**
>
> The original ADTOF repository is distributed under CC BY-NC-SA 4.0. The PyTorch port bundles converted model weights, while its current package metadata does not declare a software licence. For commercial deployment, do not assume the code and weights are permissively licensed. Obtain explicit written clarification or permission from the relevant authors before production use.

For internal evaluation and non-commercial prototyping, ADTOF-pytorch is still the preferred technical baseline. Keep the model boundary modular so that the classifier can be replaced later without changing timing, quantization, velocity or MIDI-export components.

5. Paid options without a subscription

Superior Drummer 3 Tracker

This is the recommended paid fallback. It is a mature local audio-to-MIDI workflow with manual correction controls and a perpetual desktop licence model. Its strongest use case is multitrack drum recording or component stems. A single stereo drum stem may still work, but it is not the product's ideal input, so test representative material before committing.

Klangio Drum2Notes

Use Drum2Notes as a hosted benchmark or occasional fallback. Fixed transcription-ticket options can provide pay-as-you-go access without maintaining a subscription. It offers convenient MIDI output and more detailed drum notation, but it is a black box: audio is uploaded, raw activations are unavailable, and reproducibility may change with service updates.

Criterion	Superior Drummer 3	Drum2Notes
Runs locally	Yes	No
Commercial model	Perpetual desktop licence	Pay-as-you-go tickets available
Best input	Multitrack/component drums	Uploaded song or drum audio
Raw activations/API control	No model API	No
Primary role	Production fallback/reference	External benchmark/occasional job

6. Validation plan

Before selecting a production engine, create a small internal benchmark of approximately 20-50 representative drum stems. Include sparse grooves, dense fills, ghost notes, double-kick passages, open hi-hats, rides, crashes, tom-heavy sections and multiple production styles.

- Measure precision, recall and F1 separately for kick, snare, tom, hi-hat and cymbal; do not rely only on aggregate F1.
- Evaluate onset timing before and after grid snapping.
- Count musically expensive errors separately: missing backbeats, false crashes and duplicated kick events.
- Compare ADTOF-pytorch, Superior Drummer 3 Tracker and Drum2Notes on the same files where licensing and upload constraints permit.
- Retain raw predictions and correction logs so threshold changes can be evaluated reproducibly.

Final decision

Adopt **ADTOF-pytorch raw activations + custom post-processing** as the technical baseline. Keep a classifier abstraction layer because commercial licensing of the ADTOF-derived weights is unresolved. Evaluate **Superior Drummer 3 Tracker** as the preferred perpetual paid fallback, and use **Drum2Notes** as a pay-as-you-go external comparator rather than the core engine.

References

- ADTOF-pytorch repository
- Original ADTOF repository
- Superior Drummer 3
- Klangio Drum2Notes
- ADT_STR repository

Note: benchmark figures and licensing observations are based on the public project materials reviewed for this recommendation. Commercial terms and licences should be verified directly before procurement or deployment.